

DSA STUDY BLUEPRINT SERIES

101. Populate Next Right Pointers in Each Node

Connecting Nodes at Same Depth Level

Section 09 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Link node's `next` pointer to its adjacent right neighbor.
- Perform BFS level-order or recurse using parent nodes.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
TreeNode* connect(TreeNode* root) {  
    if (!root) return nullptr;  
    TreeNode* curr = root;  
    while (curr && curr->left) {  
        TreeNode* temp = curr;  
        while (temp) {  
            temp->left->next = temp->right;  
            if (temp->next) temp->right->next = temp->next->left;  
            temp = temp->next;  
        }  
        curr = curr->left;  
    }  
    return root;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(1)$ (Iterative linkages)

Key Practices & Gotchas

- **Important:** This approach operates in-place without recursion stack allocations.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace BST constraint validation checks verifying node ranges `[Min, Max]`:

Active Node	Valid range limit	Comparison check	Recursive branch step
15 (Root)	$[-\infty, +\infty]$	True $(-\infty < 15 < +\infty)$	Recurse Left child with range $[-\infty, 15]$
10 (Left)	$[-\infty, 15]$	True $(-\infty < 10 < 15)$	Recurse Right child with range $[10, 15]$

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Inorder property:** BST Inorder traversals yield strictly increasing sorted lists.
- **Tree Balancing:** Balanced tree heights (AVL/Red-black) guarantee logarithmic search times.
- **Related LeetCode Problems:** #98 Validate Binary Search Tree, #700 Search in a Binary Search Tree, #108 Convert Sorted Array to Binary Search Tree.